

Punto 4.10 - Paginación eficiente: Skip/Take y keyset pagination

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que usen Skip/Take y keyset pagination. Comparar el SQL generado y el tiempo de ejecución. Aplicar paginación con filtros, proyecciones y AsNoTracking. Analizar el impacto de Skip en tablas grandes.

Contexto del proyecto: En el punto 4.9 se estudiaron las Compiled Queries, incluyendo su compilación y reutilización. En este punto se profundiza en la paginación eficiente, que es esencial en aplicaciones web y en cualquier escenario con muchos resultados. Esta técnica se usará en el punto 4.11 para el diagnóstico.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de paginación a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenResumenDto> ObtenerPaginadoOffset(int pagina, int tamañoPagina);
    List<OrdenResumenDto> ObtenerPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamañoPagina);
    int ContarOrdenesPaginadas();
    string ObtenerSqlPaginadoOffset(int pagina, int tamañoPagina);
    string ObtenerSqlPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamañoPagina);

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerPaginadoOffset(int pagina, int tamañoPagina); → método que usa offset pagination.

Línea 11: List ObtenerPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamañoPagina); → método que usa keyset pagination.

Línea 12: int ContarOrdenesPaginadas(); → método que cuenta las órdenes para calcular el total de páginas.

Línea 13: `string ObtenerSqlPaginadoOffset(int pagina, int tamanoPagina);` → método que devuelve el SQL de offset pagination.

Línea 14: `string ObtenerSqlPaginadoKeyset(DateTime ultimaFecha, int ultimold, int tamanoPagina);` → método que devuelve el SQL de keyset pagination.

Línea 16: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 17: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 3: Implementar los métodos de paginación en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepository.cs`:

```
public List<OrdenResumenDto> ObtenerPaginadoOffset(int pagina, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

public List<OrdenResumenDto> ObtenerPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id >
ultimoId))
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

public int ContarOrdenesPaginadas()
{
    return _context.OrdenesFabricacion.Count();
}

public string ObtenerSqlPaginadoOffset(int pagina, int tamanoPagina)
{
    var consulta = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
```

```

        Estado = o.Estado,
        FechaCreacion = o.FechaCreacion
    });

    return consulta.ToQueryString();
}

public string ObtenerSqlPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamanoPagina)
{
    var consulta = _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id >
ultimoId))
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        });

    return consulta.ToQueryString();
}

```

Línea 1: public List ObtenerPaginadoOffset(int pagina, int tamanoPagina) → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 6: .ThenBy(o => o.Id) → ordena por Id.

Línea 7: .Skip((pagina - 1) * tamanoPagina) → salta las páginas anteriores.

Línea 8: .Take(tamanoPagina) → toma el tamaño de la página.

Línea 9: .Select(o => new OrdenResumenDto → proyecta al DTO.

Línea 10: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 11: Cliente = o.Cliente, → asigna el cliente.

Línea 12: Estado = o.Estado, → asigna el estado.

Línea 13: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 14: }) → cierra la proyección.

Línea 15: .ToList(); → materializa la consulta.

Línea 18: public List ObtenerPaginadoKeyset(DateTime ultimaFecha, int ultimold, int tamanoPagina) → declara el método.

Línea 20: return _context.OrdenesFabricacion → inicia la consulta.

Línea 21: .AsNoTracking() → aplica AsNoTracking.

Línea 22: .Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id > ultimold)) → filtra por la clave compuesta.

Línea 23: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 24: .ThenBy(o => o.Id) → ordena por Id.

Línea 25: .Take(tamanoPagina) → toma el tamaño de la página.

Línea 26: `.Select(o => new OrdenResumenDto` → proyecta al DTO.

Línea 27: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 28: `Cliente = o.Cliente,` → asigna el cliente.

Línea 29: `Estado = o.Estado,` → asigna el estado.

Línea 30: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 31: `})` → cierra la proyección.

Línea 32: `.ToList();` → materializa la consulta.

Línea 35: `public int ContarOrdenesPaginadas()` → declara el método.

Línea 37: `return _context.OrdenesFabricacion.Count();` → cuenta las órdenes.

Línea 40: `public string ObtenerSqlPaginadoOffset(int pagina, int tamanoPagina)` → declara el método.

Línea 42: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 43: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 44: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 45: `.ThenBy(o => o.Id)` → ordena por Id.

Línea 46: `.Skip((pagina - 1) * tamanoPagina)` → salta las páginas anteriores.

Línea 47: `.Take(tamanoPagina)` → toma el tamaño de la página.

Línea 48: `.Select(o => new OrdenResumenDto` → proyecta al DTO.

Línea 49: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 50: `Cliente = o.Cliente,` → asigna el cliente.

Línea 51: `Estado = o.Estado,` → asigna el estado.

Línea 52: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 53: `});` → cierra la proyección.

Línea 55: `return consulta.ToQueryString();` → devuelve el SQL.

Línea 58: `public string ObtenerSqlPaginadoKeyset(DateTime ultimaFecha, int ultimold, int tamanoPagina)` → declara el método.

Línea 60: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 61: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 62: `.Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id > ultimold))` → filtra por la clave compuesta.

Línea 63: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 64: `.ThenBy(o => o.Id)` → ordena por Id.

Línea 65: `.Take(tamanoPagina)` → toma el tamaño de la página.

Línea 66: `.Select(o => new OrdenResumenDto` → proyecta al DTO.

Línea 67: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 68: `Cliente = o.Cliente,` → asigna el cliente.

Línea 69: `Estado = o.Estado,` → asigna el estado.

Línea 70: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 71: }); → cierra la proyección.

Línea 73: return consulta.ToString(); → devuelve el SQL.

Error común: si la clave de ordenación no es única, la keyset pagination puede devolver filas duplicadas o saltar filas. Se debe usar una clave compuesta que incluya la clave primaria.

| Paso 4: Crear el caso de uso de paginación eficiente

Crear el archivo src/AceriaData.Application/UseCases/PaginacionUseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class PaginacionUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public PaginacionUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== PAGINACIÓN EFICIENTE ===");

        DemostrarOffsetPagination();
        DemostrarKeysetPagination();
        CompararRendimiento();
        MostrarSql();
    }

    private void DemostrarOffsetPagination()
    {
        Console.WriteLine("\n--- Offset pagination ---");

        var cronometro = Stopwatch.StartNew();
        var pagina1 = _unidad.Ordenes.ObtenerPaginadoOffset(1, 3);
        var pagina2 = _unidad.Ordenes.ObtenerPaginadoOffset(2, 3);
        cronometro.Stop();

        Console.WriteLine($"Página 1: {pagina1.Count} órdenes | Página 2: {pagina2.Count} órdenes |
Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in pagina1)
        {
            Console.WriteLine($" P1: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/
MM/yyyy}");
        }
        foreach (var orden in pagina2)
        {
            Console.WriteLine($" P2: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/
MM/yyyy}");
        }
    }

    private void DemostrarKeysetPagination()
    {
        Console.WriteLine("\n--- Keyset pagination ---");

        var cronometro = Stopwatch.StartNew();
        var pagina1 = _unidad.Ordenes.ObtenerPaginadoKeyset(DateTime.MinValue, 0, 3);
        if (pagina1.Count > 0)
        {
            var ultima = pagina1.Last();
            var pagina2 = _unidad.Ordenes.ObtenerPaginadoKeyset(ultima.FechaCreacion, 0, 3);
            cronometro.Stop();

            Console.WriteLine($"Página 1: {pagina1.Count} órdenes | Página 2: {pagina2.Count} órdenes |
Tiempo: {cronometro.ElapsedMilliseconds} ms");
        }
    }
}
```

```

        foreach (var orden in pagina1)
        {
            Console.WriteLine($" P1: {orden.NumeroOrden} | {orden.Cliente} |
{orden.FechaCreacion:dd/MM/yyyy}");
        }
        foreach (var orden in pagina2)
        {
            Console.WriteLine($" P2: {orden.NumeroOrden} | {orden.Cliente} |
{orden.FechaCreacion:dd/MM/yyyy}");
        }
    }
}

private void CompararRendimiento()
{
    Console.WriteLine("\n--- Comparación de rendimiento ---");

    var cronometroOffset = Stopwatch.StartNew();
    for (int i = 1; i <= 10; i++)
    {
        _unidad.Ordenes.ObtenerPaginadoOffset(i, 3);
    }
    cronometroOffset.Stop();

    var cronometroKeyset = Stopwatch.StartNew();
    var ultimaFecha = DateTime.MinValue;
    var ultimoId = 0;
    for (int i = 0; i < 10; i++)
    {
        var pagina = _unidad.Ordenes.ObtenerPaginadoKeyset(ultimaFecha, ultimoId, 3);
        if (pagina.Count == 0) break;
        var ultima = pagina.Last();
        ultimaFecha = ultima.FechaCreacion;
        ultimoId = 0;
    }
    cronometroKeyset.Stop();

    Console.WriteLine($"Offset pagination (10 páginas): {cronometroOffset.ElapsedMilliseconds} ms");
    Console.WriteLine($"Keyset pagination (10 páginas): {cronometroKeyset.ElapsedMilliseconds} ms");
}

private void MostrarSql()
{
    Console.WriteLine("\n--- SQL de offset pagination ---");
    var sqlOffset = _unidad.Ordenes.ObtenerSqlPaginadoOffset(2, 3);
    Console.WriteLine(sqlOffset);

    Console.WriteLine("\n--- SQL de keyset pagination ---");
    var sqlKeyset = _unidad.Ordenes.ObtenerSqlPaginadoKeyset(new DateTime(2024, 3, 10), 3, 3);
    Console.WriteLine(sqlKeyset);
}
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class PaginacionUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public PaginacionUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== PAGINACIÓN EFICIENTE ==="); → muestra la cabecera.

Línea 19: DemostrarOffsetPagination(); → llama al método de offset pagination.

Línea 20: `DemostrarKeysetPagination();` → llama al método de keyset pagination.

Línea 21: `CompararRendimiento();` → llama al método de comparación.

Línea 22: `MostrarSql();` → llama al método de SQL.

Línea 25: `private void DemostrarOffsetPagination()` → declara el método.

Línea 27: `Console.WriteLine("\n--- Offset pagination ---");` → muestra la cabecera.

Línea 29: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 30: `var pagina1 = _unidad Ordenes.ObtenerPaginadoOffset(1, 3);` → carga la primera página.

Línea 31: `var pagina2 = _unidad Ordenes.ObtenerPaginadoOffset(2, 3);` → carga la segunda página.

Línea 32: `cronometro.Stop();` → detiene el cronómetro.

Línea 34: `Console.WriteLine($"Página 1: {pagina1.Count} órdenes | Página 2: {pagina2.Count} órdenes | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el resultado.

Línea 35: `foreach (var orden in pagina1)` → itera sobre la primera página.

Línea 37: `Console.WriteLine($" P1: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 39: `foreach (var orden in pagina2)` → itera sobre la segunda página.

Línea 41: `Console.WriteLine($" P2: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 45: `private void DemostrarKeysetPagination()` → declara el método.

Línea 47: `Console.WriteLine("\n--- Keyset pagination ---");` → muestra la cabecera.

Línea 49: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 50: `var pagina1 = _unidad Ordenes.ObtenerPaginadoKeyset(DateTime.MinValue, 0, 3);` → carga la primera página.

Línea 51: `if (pagina1.Count > 0)` → comprueba si hay resultados.

Línea 53: `var ultima = pagina1.Last();` → obtiene la última orden.

Línea 54: `var pagina2 = _unidad Ordenes.ObtenerPaginadoKeyset(ultima.FechaCreacion, 0, 3);` → carga la segunda página.

Línea 55: `cronometro.Stop();` → detiene el cronómetro.

Línea 57: `Console.WriteLine($"Página 1: {pagina1.Count} órdenes | Página 2: {pagina2.Count} órdenes | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra el resultado.

Línea 58: `foreach (var orden in pagina1)` → itera sobre la primera página.

Línea 60: `Console.WriteLine($" P1: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 62: `foreach (var orden in pagina2)` → itera sobre la segunda página.

Línea 64: `Console.WriteLine($" P2: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 69: `private void CompararRendimiento()` → declara el método.

Línea 71: `Console.WriteLine("\n--- Comparación de rendimiento ---");` → muestra la cabecera.

Línea 73: `var cronometroOffset = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 74: `for (int i = 1; i <= 10; i++)` → repite diez veces.

Línea 76: `_unidad Ordenes.ObtenerPaginadoOffset(i, 3);` → carga la página.

Línea 78: `cronometroOffset.Stop();` → detiene el cronómetro.

Línea 80: `var cronometroKeyset = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 81: `var ultimaFecha = DateTime.MinValue;` → inicializa la fecha.

Línea 82: `var ultimold = 0;` → inicializa el Id.

Línea 83: `for (int i = 0; i < 10; i++)` → repite diez veces.

Línea 85: `var pagina = _unidad Ordenes.ObtenerPaginadoKeyset(ultimaFecha, ultimold, 3);` → carga la página.

Línea 86: `if (pagina.Count == 0) break;` → detiene si no hay más páginas.

Línea 87: `var ultima = pagina.Last();` → obtiene la última orden.

Línea 88: `ultimaFecha = ultima.FechaCreacion;` → actualiza la fecha.

Línea 89: `ultimold = 0;` → reinicia el Id.

Línea 91: `cronometroKeyset.Stop();` → detiene el cronómetro.

Línea 93: `Console.WriteLine($"Offset pagination (10 páginas): {cronometroOffset.ElapsedMilliseconds} ms");` → muestra el tiempo de offset.

Línea 94: `Console.WriteLine($"Keyset pagination (10 páginas): {cronometroKeyset.ElapsedMilliseconds} ms");` → muestra el tiempo de keyset.

Línea 97: `private void MostrarSql()` → declara el método.

Línea 99: `Console.WriteLine("\n--- SQL de offset pagination ---");` → muestra la cabecera.

Línea 100: `var sqlOffset = _unidad Ordenes.ObtenerSqlPaginadoOffset(2, 3);` → obtiene el SQL.

Línea 101: `Console.WriteLine(sqlOffset);` → imprime el SQL.

Línea 103: `Console.WriteLine("\n--- SQL de keyset pagination ---");` → muestra la cabecera.

Línea 104: `var sqlKeyset = _unidad Ordenes.ObtenerSqlPaginadoKeyset(new DateTime(2024, 3, 10), 3, 3);` → obtiene el SQL.

Línea 105: `Console.WriteLine(sqlKeyset);` → imprime el SQL.

Error común: si se usa el mismo `DbContext` para todas las páginas, las entidades permanecen en el `Change Tracker`. Se debe usar `AsNoTracking` para evitar el consumo de memoria.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<PaginacionUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<PaginacionUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

▮ Paso 7: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>();
    for (int i = 1; i <= 20; i++)
    {
        ordenes.Add(new OrdenFabricacion
        {
            NumeroOrden = $"OF-2024-{i:D4}",
            Cliente = i % 2 == 0 ? "Constructora del Norte" : "Constructora del Sur",
            Estado = i % 3 == 0 ? "EnProceso" : "Pendiente",
            FechaCreacion = new DateTime(2024, 1, 1).AddDays(i)
        });
    }

    context OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base de datos.

Línea 5: `context.Database.Migrate();` → aplica las migraciones.

Línea 7: `var ordenes = new List();` → crea la lista de órdenes.

Línea 8: `for (int i = 1; i <= 20; i++)` → repite veinte veces.

Línea 10: `ordenes.Add(new OrdenFabricacion` → crea una orden.

Línea 12: `NumeroOrden = $"OF-2024-{i:D4}"`, → asigna el número.

Línea 13: `Cliente = i % 2 == 0 ? "Constructora del Norte" : "Constructora del Sur"`, → asigna el cliente.

Línea 14: `Estado = i % 3 == 0 ? "EnProceso" : "Pendiente"`, → asigna el estado.

Línea 15: `FechaCreacion = new DateTime(2024, 1, 1).AddDays(i)` → asigna la fecha.

Línea 16: `});` → cierra la orden.

Línea 17: `}` → cierra el bucle.

Línea 19: `context OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes.

Línea 20: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo de offset pagination y keyset pagination, y el SQL generado por cada técnica.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== PAGINACIÓN EFICIENTE ===

--- Offset pagination ---
Página 1: 3 órdenes | Página 2: 3 órdenes | Tiempo: 48 ms
P1: 0F-2024-0001 | Constructora del Sur | 02/01/2024
P1: 0F-2024-0002 | Constructora del Norte | 03/01/2024
P1: 0F-2024-0003 | Constructora del Sur | 04/01/2024
P2: 0F-2024-0004 | Constructora del Norte | 05/01/2024
P2: 0F-2024-0005 | Constructora del Sur | 06/01/2024
P2: 0F-2024-0006 | Constructora del Norte | 07/01/2024

--- Keyset pagination ---
Página 1: 3 órdenes | Página 2: 3 órdenes | Tiempo: 45 ms
P1: 0F-2024-0001 | Constructora del Sur | 02/01/2024
...

--- Comparación de rendimiento ---
Offset pagination (10 páginas): 120 ms
Keyset pagination (10 páginas): 95 ms

--- SQL de offset pagination ---
SELECT [o].[NumeroOrden], [o].[Cliente], [o].[Estado], [o].[FechaCreacion]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[FechaCreacion], [o].[Id]
OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY

--- SQL de keyset pagination ---
SELECT [o].[NumeroOrden], [o].[Cliente], [o].[Estado], [o].[FechaCreacion]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
      AND ([o].[FechaCreacion] > @__ultimaFecha_0 OR ([o].[FechaCreacion] = @__ultimaFecha_0 AND [o].[Id]
> @__ultimoId_1))
ORDER BY [o].[FechaCreacion], [o].[Id]
OFFSET 0 ROWS FETCH NEXT 3 ROWS ONLY
```

La primera sección muestra el tiempo de offset pagination. La segunda sección muestra el tiempo de keyset pagination. La tercera sección muestra la comparación. La cuarta sección muestra el SQL de offset pagination. La quinta sección muestra el SQL de keyset pagination.

Observaciones: keyset pagination es más rápida porque no lee las filas anteriores. El SQL de offset pagination incluye OFFSET 3. El SQL de keyset pagination incluye el filtro por la clave compuesta y OFFSET 0. La diferencia es pequeña con pocas filas, pero crece con el número de filas.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerPaginadoKeyset para usar solo la fecha como clave:

```

public List<OrdenResumenDto> ObtenerPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.FechaCreacion > ultimaFecha)
        .OrderBy(o => o.FechaCreacion)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion \           })
        .ToList();
}

```

Resultado esperado: el código funciona, pero si dos órdenes tienen la misma fecha, la paginación puede saltar filas o devolver filas duplicadas. La clave de ordenación no es única.

Solución: usar una clave compuesta que incluya la fecha y el Id.

```

public List<OrdenResumenDto> ObtenerPaginadoKeyset(DateTime ultimaFecha, int ultimoId, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id >
ultimoId))
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Take(tamanoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

```

Resultado esperado con la solución: la paginación devuelve las filas correctas aunque haya fechas duplicadas.

Errores comunes del ejercicio

Error	Causa	Solución
Skip costoso	Se salta un número elevado de filas	Usar keyset pagination
Filas duplicadas	La clave de ordenación no es única	Usar clave compuesta
Filas saltadas	La clave de ordenación no es única	Usar clave compuesta
Total de páginas incorrecto	El conteo no usa los mismos filtros	Usar los mismos filtros
Paginación sin orden	Se omite OrderBy	Aplicar OrderBy antes de Skip
Paginación en memoria	Se materializa antes de paginar	Aplicar Skip y Take antes de ToList

Reto resuelto: Paginación con filtro y proyección

Reto: Crear un método en el repositorio que pague las órdenes pendientes ordenadas por fecha, con proyección a DTO y AsNoTracking. Comparar el SQL generado con y sin filtro.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```
List<OrdenResumenDto> ObtenerPendientesPaginado(int pagina, int tamañoPagina);
```

Línea 1: List ObtenerPendientesPaginado(int pagina, int tamañoPagina); → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```
public List<OrdenResumenDto> ObtenerPendientesPaginado(int pagina, int tamañoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Skip((pagina - 1) * tamañoPagina)
        .Take(tamañoPagina)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}
```

Línea 1: public List ObtenerPendientesPaginado(int pagina, int tamañoPagina) → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .Where(o => o.Estado == "Pendiente") → filtra por estado.

Línea 6: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 7: .ThenBy(o => o.Id) → ordena por Id.

Línea 8: .Skip((pagina - 1) * tamañoPagina) → salta las páginas anteriores.

Línea 9: .Take(tamañoPagina) → toma el tamaño de la página.

Línea 10: .Select(o => new OrdenResumenDto → proyecta al DTO.

Línea 11: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 12: Cliente = o.Cliente, → asigna el cliente.

Línea 13: Estado = o.Estado, → asigna el estado.

Línea 14: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 15: }) → cierra la proyección.

Línea 16: .ToList(); → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarPaginadoConFiltro()
{
    Console.WriteLine("\n--- Paginación con filtro y proyección ---");

    var pagina1 = _unidad.Ordenes.ObtenerPendientesPaginado(1, 3);
    var pagina2 = _unidad.Ordenes.ObtenerPendientesPaginado(2, 3);

    Console.WriteLine($"Página 1: {pagina1.Count} órdenes | Página 2: {pagina2.Count} órdenes");
    foreach (var orden in pagina1)
    {
        Console.WriteLine($" P1: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");
    }
    foreach (var orden in pagina2)
    {
        Console.WriteLine($" P2: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");
    }
}
```

Línea 1: private void DemostrarPaginadoConFiltro() → declara el método.

Línea 3: Console.WriteLine("\n--- Paginación con filtro y proyección ---"); → muestra la cabecera.

Línea 5: var pagina1 = _unidad.Ordenes.ObtenerPendientesPaginado(1, 3); → carga la primera página.

Línea 6: var pagina2 = _unidad.Ordenes.ObtenerPendientesPaginado(2, 3); → carga la segunda página.

Línea 8: Console.WriteLine(\$"Página 1: {pagina1.Count} órdenes | Página 2: {pagina2.Count} órdenes"); → muestra el resultado.

Línea 9: foreach (var orden in pagina1) → itera sobre la primera página.

Línea 11: Console.WriteLine(\$" P1: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}"); → muestra los datos.

Línea 13: foreach (var orden in pagina2) → itera sobre la segunda página.

Línea 15: Console.WriteLine(\$" P2: {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}"); → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarPaginadoConFiltro();
```

Paso 5: Ejecutar dotnet run y verificar que las páginas se muestran correctamente.

Resultado esperado: las órdenes pendientes se muestran paginadas por fecha, con proyección a DTO y sin tracking.

Analogía final

La paginación en una acería es como pedir las carpetas al archivo central en lotes. En lugar de pedir todas las carpetas de una vez, se piden de diez en diez. La offset pagination es como pedir el lote número diez: el archivo tiene que contar las carpetas anteriores y saltarlas. Si se pide el lote número mil, el archivo tiene que contar mil lotes antes de llegar al solicitado. La keyset pagination es como pedir el lote que empieza después de la carpeta con número OF-2024-0100: el archivo va directamente a esa carpeta y trae las siguientes. No tiene que contar las anteriores. La keyset pagination es más eficiente porque no recorre las carpetas anteriores. La paginación con filtro es como pedir solo las carpetas de un estado concreto. La paginación con proyección es como pedir solo los datos necesarios de cada carpeta.

Así funciona la paginación en EF Core: se elige la técnica según el escenario y se combina con filtros y proyecciones.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de paginación a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `PaginacionUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con veinte órdenes.

Ejecutado las demostraciones de `offset pagination` y `keyset pagination`.

Comparado el rendimiento de ambas técnicas.

Analizado el SQL generado por cada técnica.

Diagnosticado el error de usar solo la fecha como clave.

Creado el método `ObtenerPendientesPaginado` con filtro y proyección.
